

Peer-to-Peer Protocols for Efficient and Resilient Decentralised Learning

Theory, Design and Evaluation



Mohamed Amine LEGHERABA

PhD Defence · 2025



Supervised by Pr. Maria Potop-Butucaru & Pr. Sébastien Tixeuil

Opening



Mohamed Amine LEGHERABA

- **2018**: Engineering degree in Computer Science
- **2018-2022**: 4 years in IT consulting
 - Blockchain architecture
 - DevOps engineering
- **Feb. 2023**: Started PhD at Sorbonne Université, LIP6
 - Supervisors: Maria Potop-Butucaru, Sébastien Tixeuil
 - Team: NPA (Networks, Performance and Algorithms)

**3-month research
stay**

National Institute
of Informatics
Tokyo, Japan

Host: Megumi
Kaneko



- **Initial plan:** Security of FL (privacy attacks, poisoning attacks) using blockchain
- **Realisation:** Blockchain consensus $\neq$ ML optimisation — mismatch of objectives
- **Discovery:** Gossip learning already eliminates the central server, but convergence is slow
- **Key insight:** The bottleneck is the **communication topology**, not the learning algorithm

This thesis is part of the **NEMO** project
(**N*****e**tworks of **E***volving **M*****i**cro-**O***perators)

- European project (H2020)
- Rethinking Internet and 5G network architectures in Europe
- Goal: decentralised, micro-operator driven networks
- Key challenge: distributed coordination without central authority
- Our contribution: peer-to-peer protocols for decentralised learning

From security to performance

We initially targeted security (privacy, poisoning)

→ Realised gossip learning performance was the bottleneck

→ Shifted focus to peer sampling & overlay topology

Modern AI development follows a structural tendency toward **centralisation**:

- A **small number** of actors collect the data, train the models, deploy the services
- Users interact through opaque APIs — no access to data, code, or training process
- The **black box problem**: models whose decisions cannot be traced to interpretable rules

Three interrelated concerns:

Privacy

Raw data leaves the hands of those who generated it

Sovereignty

Dependency on private actors for critical infrastructure

Accountability

Opaque models resist democratic oversight

Use case 1: Healthcare

A hospital with a rare disease has too few cases to train alone — neighbouring hospitals face the same constraint, yet patient data cannot be shared.

Use case 2: Autonomous vehicles

Manufacturers guard their data jealously, yet all share an interest in reducing accidents — a model that improves with collective data, produced by a process that compromises no one.

Use case 3: Language models

A model trained collectively by heterogeneous participants would be structurally more resistant to commercial, ideological, or political influence.

Decentralised learning: privacy and the absence of central control are structural consequences of how the system is designed.

Can we build a decentralised learning system that matches the efficiency of centralised federated learning, while being fully decentralised and resilient to failures?

Foundations

C1: Elevator

Decentralised overlay protocol that organises nodes into a hub-and-spoke topology through emergent hub election

C2: Lift

Byzantine-resilient extension of Elevator that prevents adversarial nodes from manipulating hub election

C3: HEAL

Decentralised federated learning framework built on Elevator — hub-based aggregation recovers FL efficiency

C4: FLAIR

Adaptation to wireless networks using LEACH-style clustering — validates the layered architecture

Node — abstract computational entity:

State machine (S, s_0, δ) , partial view $P(v) \subseteq V$, cache size $c \ll N$

Overlay network — directed graph $G = (V, E)$:

Vertices = nodes, edges = communication channels (unidirectional or bidirectional)

Protocol — distributed algorithm executed by all nodes:

Specifies local state, messages, transition rules. All nodes share global parameters (c, h) .

Execution model:

- **Protocol step** = one execution of the protocol loop by one node
- **Protocol cycle** = every node executes one step (synchronous or sequential)
- **Time-varying graph** $G(t) = (V(t), E(t))$ to capture churn

Global state = concatenation of all local states: $S_{\text{global}} = (s_v)_{v \in V}$

Crash failures

- Node permanently stops
- No message sent/received
- Model: CSMP $\langle n, t \rangle [\emptyset]$
- Permanent, no recovery

Churn

- Dynamic arrivals/departures
- Fraction of nodes replaced per cycle
- Modelled as time-varying graph
- Period of churn + stabilisation

Byzantine failures

- Arbitrary behaviour
- Can send incorrect messages
- Can collude with others
- Model: BSMP $\langle n, t \rangle [\emptyset]$

State of the Art



Structured overlays (DHTs)

- Chord, Kademlia, Pastry
- Logarithmic diameter ✓
- Deterministic routing ✓
- Fragile under churn ✗
- Vulnerable to Byzantine ✗
- Application-specific ✗
- No hub election ✗

Unstructured overlays (gossip)

- Cyclon, Newscast, HyParView
- Resilient to churn ✓
- Low overhead ✓
- Application-agnostic ✓
- Slow convergence ✗
- No global aggregation ✗
- No hub election ✗

Neither family supports deliberate hub election for structured aggregation.

Protocols: Phenix, Gia, SG-1, T-MAN, VICINITY, Park et al.

Key insight: degree heterogeneity → hubs emerge naturally → faster dissemination

But:

- Hubs are **passive structural properties** — not coupled to any active protocol role
- Hub identity changes continuously — unreliable for learning layer
- Vulnerable to targeted attacks (removing hubs fragments the network)
- Many protocols require continuous network growth or bootstrap servers
- Degree caps in some protocols **explicitly prevent** hub formation

Topological emergence of hubs $\neq$ structured aggregation mechanism.

What is needed: a protocol that **elects** hubs, **assigns** them aggregation responsibility, and **maintains** this structure under failures.

Federated Learning

Fast convergence ✓
Star topology
Single point of failure ✗
No churn resilience ✗

Gossip Learning

Fully decentralised ✓
Churn resilient ✓
Slow convergence ✗
No aggregation ✗

Epidemic Learning

Fully decentralised ✓
Broadcast-based
Slow convergence ✗
High overhead ✗

Gap: no protocol simultaneously achieves decentralisation, scalability, and aggregation efficiency

How can the efficiency of gossip learning be improved to approach federated learning, without sacrificing decentralisation?

Our answer: the bottleneck is **not at the learning layer** — it is **one level below**, at the **peer sampling layer**. By shaping the communication topology to introduce structured aggregation, we recover FL efficiency without any central coordinator.

Elevator & Lift

Combine two attachment mechanisms:

Preferential attachment

Connect to the most connected nodes in the 2-hop neighbourhood

- hubs emerge organically
- low diameter

Random attachment

Maintain $c - h$ connections to random peers via hubs

- robustness to failures
- self-healing when hubs fail

Result: a network with h hubs where:

- Every node is at most 2 hops from any other (diameter = 2)
- Hubs emerge **spontaneously** — no central coordinator
- The number of hubs h is a **tunable parameter**

Definition (Hub): A node $h \in V$ is a **hub** if its identifier appears in the partial view of **every** node in the network:

$$\forall v \in V, \quad h \in P(v)$$

Definition (Hub Sampling Service): A protocol that, starting from an arbitrary overlay, induces the emergence of a subset $H \subseteq V$, $|H| = h$, where every node in H satisfies the hub property.

Ideally: H is drawn uniformly at random over all $\binom{|V|}{h}$ subsets.

Key property: A hub is known to **all** nodes — it can serve as:

- Aggregation point (like a server in FL)
- Source of randomness (random peer from a hub = uniform sample)

Parameters: cache size c , number of preferential connections h , backward buffer size

At each cycle, every node executes:

1. **Retrieve 2-hop neighbours:** collect caches of all c neighbours
2. **Build frequency map:** count occurrences of each node identifier
3. **Select preferred nodes:** top h most frequent $\rightarrow$ these are the **hub candidates**
4. **Request backward lists:** ask each preferred node for its incoming connections
5. **Reset cache:** clear current outgoing connections
6. **Fill cache:** h preferred + $(c - h)$ random peers from backward lists
7. **Complete if needed:** fill remaining slots from frequency map

Result: after a few cycles, the same h nodes appear in every node's cache $\rightarrow$ **hubs have emerged**

Connectivity

Overlay remains connected despite churn and topology adaptations

Convergence

From arbitrary initial state → stable hub topology in $O(\log N)$ cycles

Robustness

Resilient to crash failures, churn, and targeted hub attacks

Low diameter

Hubs reduce communication distance → diameter $l = 2$ after convergence

Stability

Once converged, hub set remains unchanged with high probability

Proposition: Once Elevator has converged to h hubs, it remains stable.

Sketch of proof:

- Number of hubs **cannot exceed** h : each node selects at most h preferred nodes
- Number of hubs **cannot decrease**: probability of a random node replacing a hub: $P_{\text{new hub}} \approx N(N - h) \left(\frac{c-h}{N} \right)^c$ This is negligible for $c \ll N$ and $h \ll N$
- Hub set **cannot change**: intersection probability of random subsets: $p(0) = q(0) - q(1) \approx 1$

Key insight: the stability is a consequence of the **concentration of indegree** on the hub nodes — random fluctuations are exponentially unlikely to perturb it.

Proposition: Starting from a random k -out graph, Elevator converges w.h.p. to exactly h hubs.

Proof structure:

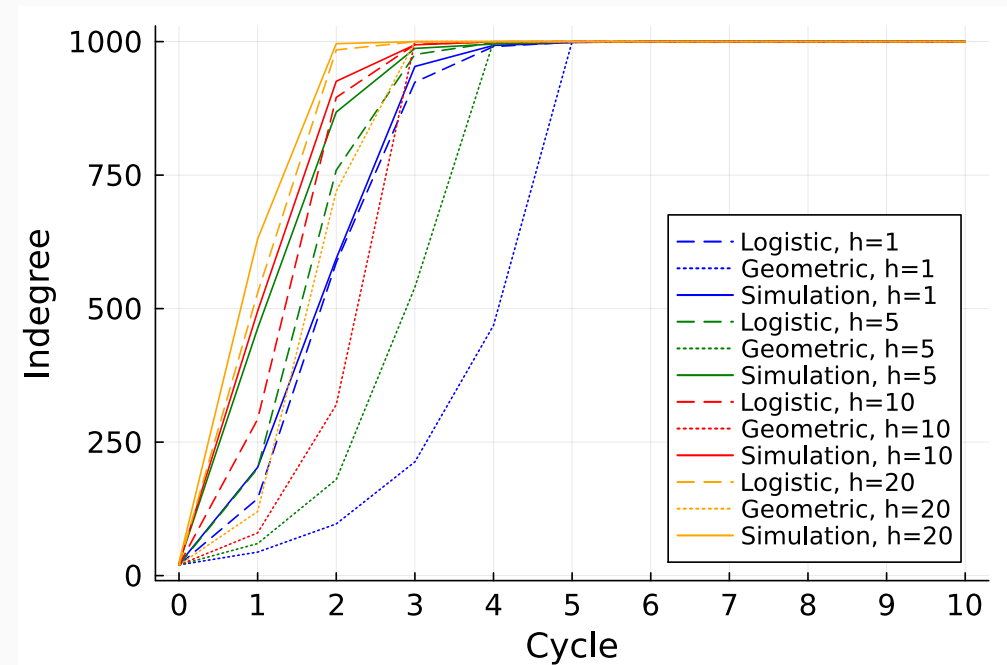
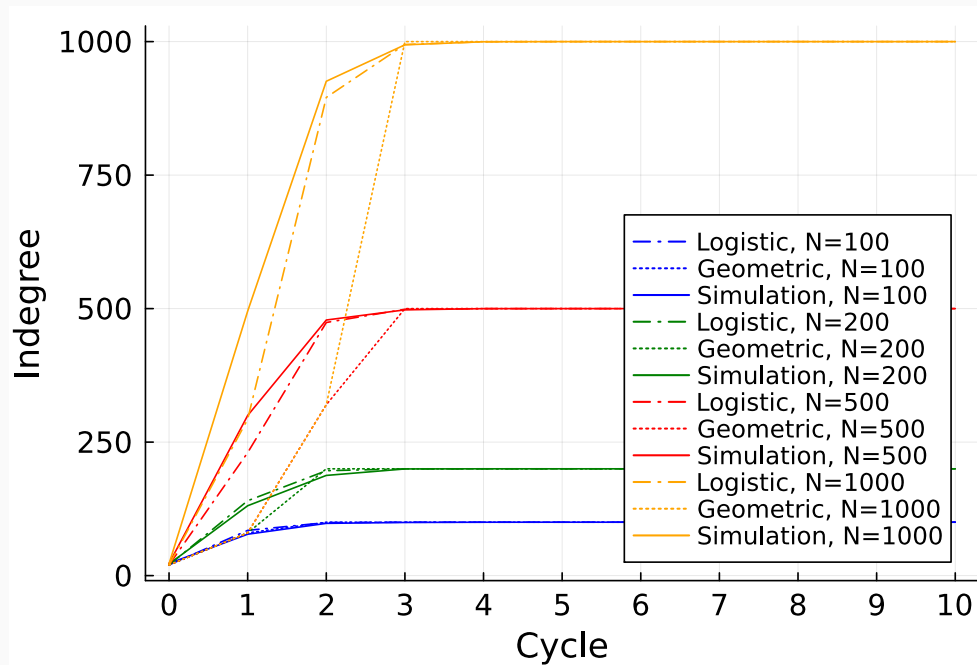
1. With sufficiently large c , at least one hub emerges after finite cycles (w.h.p.)
2. If $g = 1$ hub exists $\rightarrow$ network is strongly connected (w.h.p.)
3. Additional hubs appear eventually (non-zero probability per cycle)
4. Number of hubs is non-decreasing between 1 and h
5. Therefore: $1 \rightarrow 2 \rightarrow \dots \rightarrow h$ hubs (monotone progression)

Convergence time: modelled via a logistic function with dynamic growth rate:

$$d(t) = \frac{N}{1 + a \cdot \exp\left(-\left(r_0 t + \left(\frac{1}{2}\right) \delta_r t^2\right)\right)}$$

where $r_0 = 2 + \frac{h}{10}$, $\delta_r = \frac{K}{N}$, $a = \frac{N}{K} - 1$

Convergence Time: Model vs Simulation



Logistic model closely fits simulation data (lower RMSE than geometric model).
Convergence in 4-5 cycles for $N = 1000$ — very fast in practice.

Setup: PeerSim simulator, networks of 100 to 1000 nodes, $c = 20$, $h = 10$

Metrics compared against baselines (Newscast, Cyclon, Phenix, SimpleNewscast):

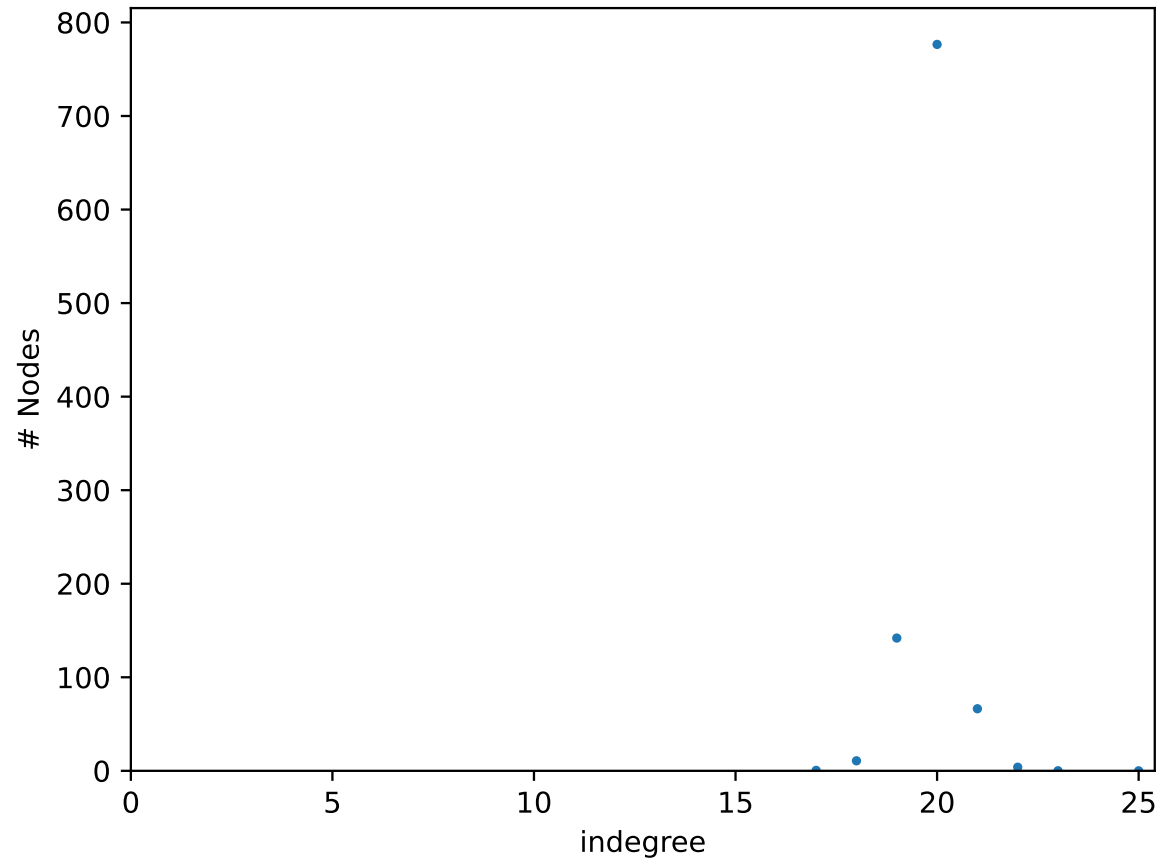
- Indegree / outdegree distribution
- Clustering coefficient
- Average path length
- Network diameter
- Largest strongly connected component

Scenarios:

- Normal (fault-free)
- Crash failures (20% of nodes)
- Crash of hub nodes

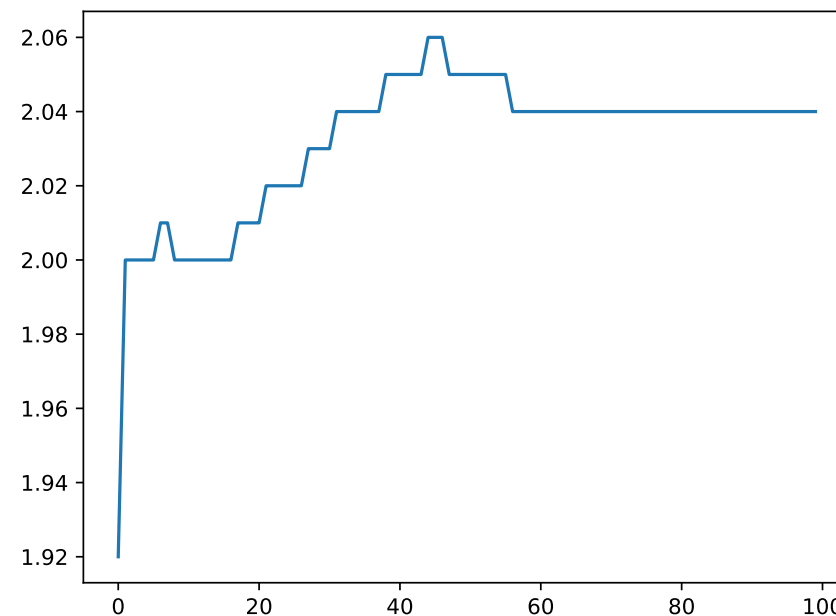
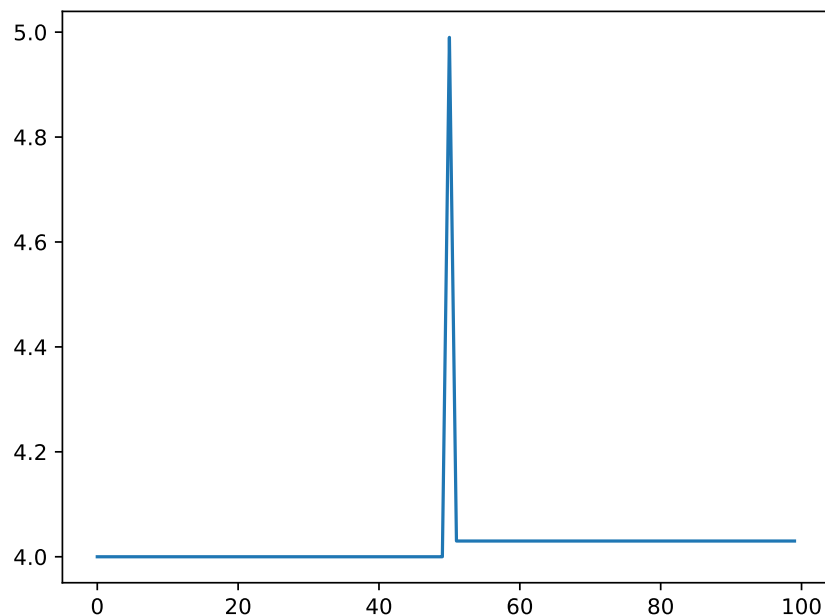
- Churn (10% per cycle)

Baselines: Newscast, Cyclon, Phenix, FedLay, SimpleNewscast



Elevator converges to a **bimodal indegree distribution**: h hubs with indegree $\approx N$, remaining nodes with indegree $\approx c - h$.

Diameter drops to 2 within 4–5 cycles.



- **Crash**: diameter remains low; new hubs are elected within 2 cycles
- **Churn**: overlay self-heals; topology stabilises quickly after churn ends
- **Hub crash**: even when all hubs fail simultaneously, the network recovers

Setup: Real network deployment over TCP/IP

- Validates that Elevator works beyond simulation
- Measures: latency of reconfiguration, message volume
- Confirms: hub emergence and stability behave as predicted

Three-level evaluation: Theory + Simulation + Real deployment
→ Consistent results across all three

Vulnerability: Elevator assumes honest reporting → Byzantine nodes can manipulate hub election

Attack types studied:

1. **Passive:** send empty cache
2. **Active:** inflate own presence in cache
3. **Non-coordinating:** each Byzantine promotes itself
4. **Coordinated:** Byzantines promote each other

Key finding: Types 1–3 are tolerated by Elevator without modification.

But: Type 4 (coordinated) — as few as **2%** of malicious nodes can guarantee all Byzantines become hubs.

The hub election is the highest-value attack surface in the entire stack.

Idea: After Elevator converges, **deterministically redistribute** hubs using a shared PRNG.

Protocol (2 phases):

Phase 1: Run standard Elevator $\rightarrow$ hub set $\mathbf{H}$ (may contain Byzantines)

Phase 2: All correct nodes execute:

1. Retrieve IDs of the $\mathbf{h}$ current hubs (identical across all nodes)
2. Build a **seed** by concatenating sorted hub IDs $\rightarrow$ same seed for all
3. Initialise PRNG with this seed $\rightarrow$ identical random sequence
4. Generate $\mathbf{h}$ new node IDs from PRNG $\rightarrow$ **new hub set**
5. Replace the first $\mathbf{h}$ cache entries with the new hubs

Result: Even if Byzantines dominated the initial hub set, the redistribution treats all nodes equally — each node has probability $\frac{h}{N}$ of becoming a hub, regardless of Byzantine status.

Tolerance: Collusion threshold extended from 2% → **10%** of Byzantine nodes.

HEAL & FLAIR



FL

Fast convergence
Central server
Single point of failure

Gossip

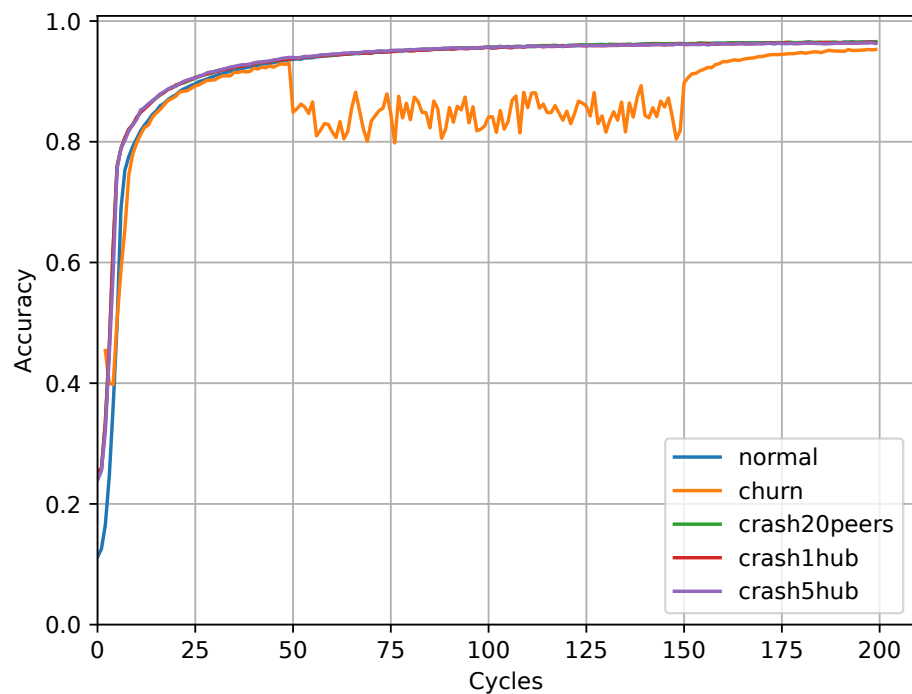
Fully decentralised
Churn resilient
Slow convergence

HEAL

FL convergence speed

- 1. Full decentralisation**
- 2. Churn & crash resilience**

Key idea: Use Elevator's hubs as **distributed aggregators** → recover the communication structure of FL without any central server.



Application Layer: ML model (SVM, NN, ...)

Aggregation Layer: FedAvg at hub level

Key design principles:

- Strict **separation of concerns**
- Each layer independent
- Well-defined interfaces between layers
- A layer can be **substituted** without affecting others
- This is validated by FLAIR (different overlay + network)

HEAL = FL aggregation + Elevator overlay

Overlay Layer: Elevator (hub election)

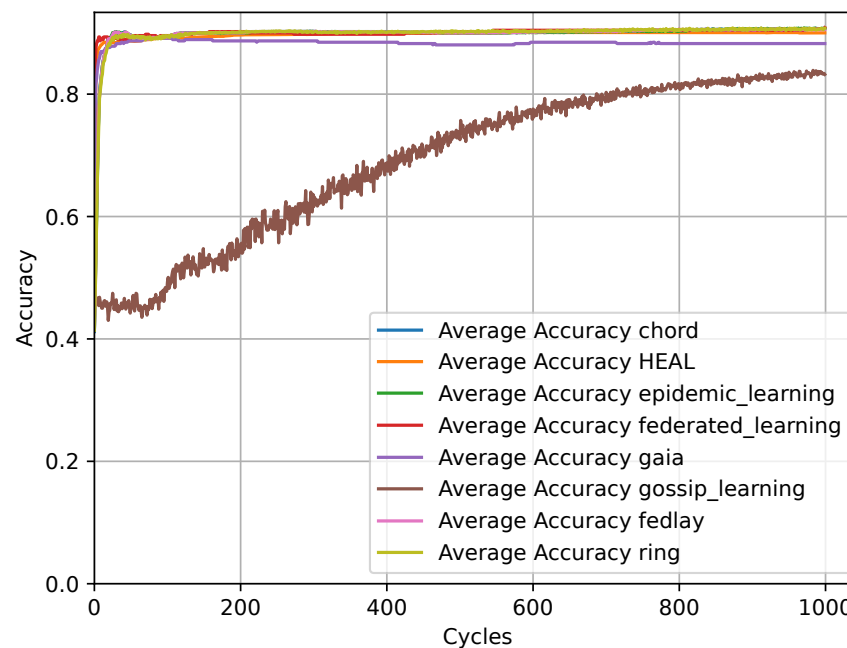
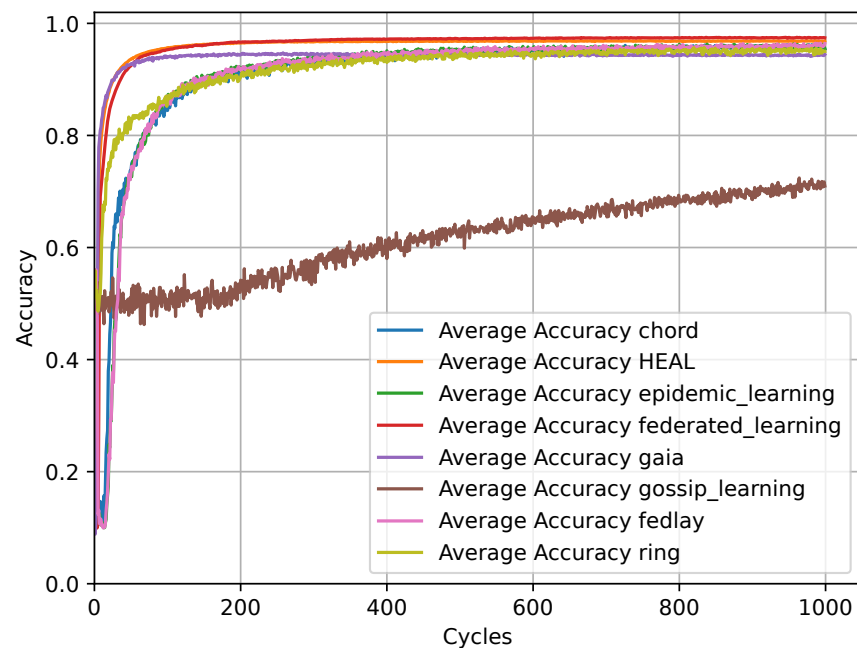
Network Layer: TCP/IP

Each cycle consists of five successive phases:

- 1 **Local training**: each node trains its model on private data
- 2 **Model transfer**: each node sends its model to s randomly chosen hubs
- 3 **Hub aggregation**: each hub averages received models (Average SGD)
- 4 **Inter-hub coordination**: hubs exchange aggregates → identical global model
- 5 **Redistribution**: hubs send global model back to contributing nodes

Communication overhead per cycle: $2(n - h) \cdot s + h(h - 1)$ messages

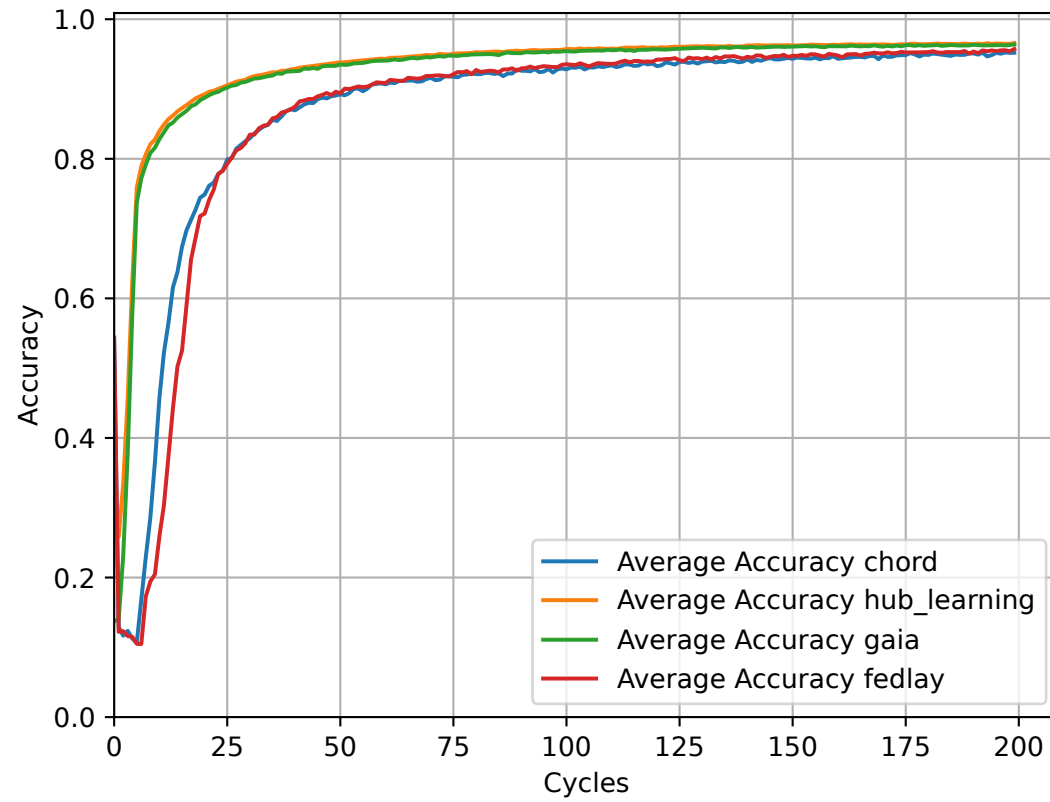
For $n = 100, h = 5, s = 1$: **210 messages** (vs 198 for FL, 1000 for Epidemic)



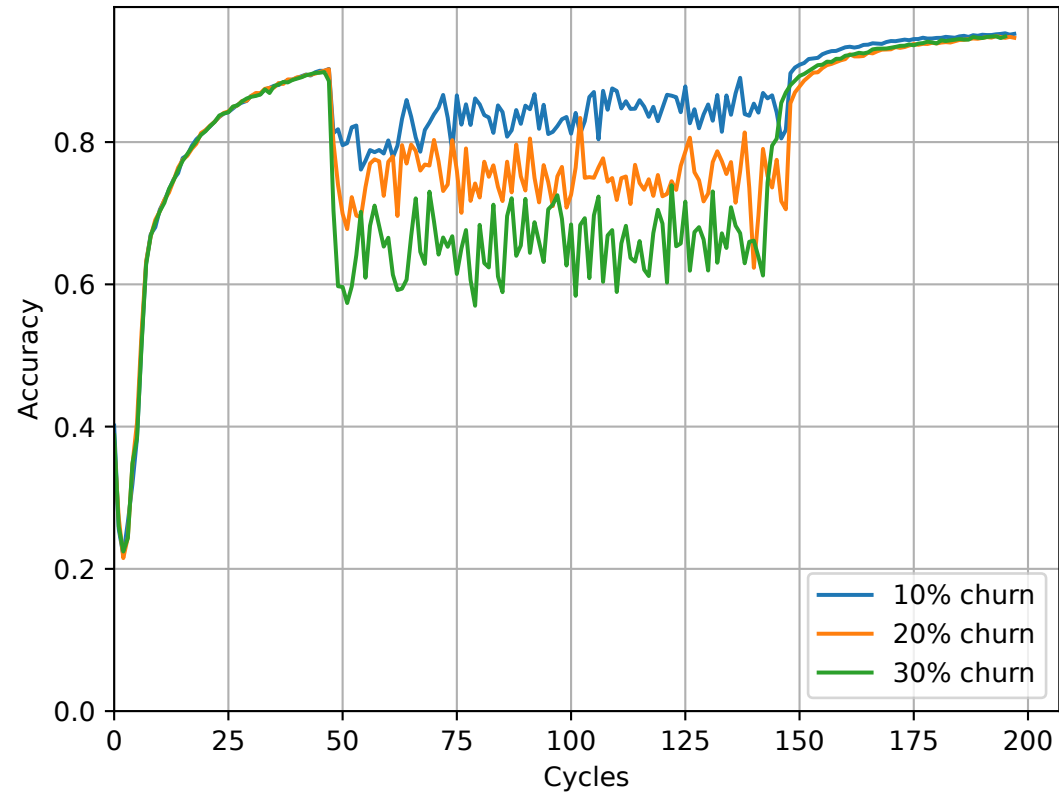
- MNIST (LeNet5): HEAL reaches 0.97 — comparable to FL (0.97), far above Gossip (0.71)
- Spambase (LogReg): HEAL reaches 0.90 — comparable to FL (0.91)
- Convergence speed: HEAL reaches 0.95 on MNIST in **76 cycles** (FL: 91 cycles)

Method	0.85	0.90	0.95
Federated Learning	22	37	91
Gaia (5 servers)	13	28	88
Gossip Learning	N/A	N/A	N/A
Epidemic Learning	90	137	372
Ring	74	157	569
Chord	98	159	451
FedLay	89	136	422
HEAL (5 hubs, $s = 1$)	15	27	76
HEAL (7 hubs, $s = 3$)	5	10	33

Cycles to reach target accuracy on MNIST. N/A = not reached within 1000 cycles.



- 20% crash at cycle 10: HEAL maintains accuracy ≈ 0.97
- **Crucially**: crash of all 5 hubs $\rightarrow$ **no measurable impact** (new hubs elected within 2 cycles)
- HEAL remains functional up to 50% crash rate
- FL and Gaia: server crash $\rightarrow$ training halts unconditionally



- 10–30% churn (cycles 50–150): temporary accuracy drop during churn phase
- **Recovery**: accuracy returns to pre-churn level almost immediately after churn ends
- At 30% churn: mean drop = 0.28 during churn, but **final accuracy = 0.95**
- FedLay: smaller drop during churn but same final accuracy

Property	FL	Gossip	HEAL
Fast convergence	✓	✗	✓
Decentralised	✗	✓	✓
Crash resilient	✗	✓	✓
Churn resilient	✗	✓	✓
No single point of failure	✗	✓	✓
Hub-targeted attack resilient	—	✓	✓
Low communication overhead	✓	✓	✓

HEAL closes the gap between FL and gossip learning.

Question: Can the layered architecture be adapted to a fundamentally different network context?

Target: Resource-constrained ad-hoc **wireless networks** (WiFi 802.11)

Key differences from HEAL:

- Overlay topology shaped by **radio range**, not explicit election
- Nodes are **heterogeneous** (CPU, RAM, bandwidth)
- Communication is **single-hop** within clusters
- No inter-cluster coordination
- Cluster heads elected based on **resource score**
- CHs **rotate** every round
- Global model mixing emerges from rotation alone
- Evaluated on **ns-3** simulator

FLAIR = existence proof that the layered architecture is modular: substitute Overlay + Network layers → working protocol for wireless edge.

Architecture:

- Network Layer: IEEE 802.11 ad-hoc
- Overlay Layer: LEACH-inspired clustering with resource-aware CH election
- Aggregation Layer: in-cluster FedAvg (no inter-CH exchange)
- Application Layer: logistic regression

CH election:

- Verifiable Random Function (VRF) for fairness
- Resource score: $R_n = \alpha \cdot \text{CPU}_n + \beta \cdot \text{RAM}_n + \gamma \cdot \text{GPU}_n + \delta \cdot \text{BW}_n$
- Threshold: $T(n) = \frac{p \cdot R_n}{1 - p \cdot (r \bmod \frac{1}{p})}$

Evaluation (ns-3):

- Binary classification on Spambase and Plants dataset
- Scenarios: fault-free, random faults, recurring faults, permanent faults, node mobility
- Baselines: LEACH, Smart-LEACH, standard FL

Key results:

- Convergence comparable to centralised FL in static settings
- Resilient to permanent and recurring faults

- Mobility has limited impact on convergence
- Resource-aware election improves over random LEACH

Conclusions & Outlook

- C1** **Elevator**: first decentralised overlay protocol with **deliberate hub emergence** — application-agnostic, $O(\log N)$ convergence, validated by theory + simulation + real deployment
- C2** **Lift**: Byzantine-resilient extension — deterministic hub redistribution via shared PRNG, extends collusion tolerance from 2% to 10%
- C3** **HEAL**: first **cross-layer** decentralised learning framework — hub-based aggregation recovers FL efficiency while preserving full decentralisation and fault resilience
- C4** **FLAIR**: validates architectural modularity — adaptation to wireless networks via layer substitution

- **Theoretical**: convergence proofs rely on idealised assumptions; no formal convergence guarantee for HEAL
- **Scale**: Elevator validated at 1 000 nodes; HEAL at 100 nodes — below P2P community standards
- **ML benchmarks**: LeNet5/MNIST, Spambase — below current standards
- **Statistical rigour**: only 5 runs per configuration; no error bars
- **IID assumption**: all experiments assume homogeneous data distributions
- **Byzantine**: Lift protects overlay only; learning layer vulnerable to poisoning
- **Synchronisation**: inter-hub phase is synchronous — tension with full decentralisation
- **FLAIR**: limited to binary classification; no convergence guarantee under non-stationary topology

- 1 **Capability-aware hub election**: bias election toward resource-rich nodes while preserving decentralisation
- 2 **Non-IID data**: adapt aggregation strategy (personalisation, locally-weighted aggregation)
- 3 **Unified simulator**: Python-based, combining peer sampling + dynamic topology + ML in a single framework

- 4 **Byzantine-hardened HEAL**: robust aggregation + differential privacy + secure aggregation at the learning layer
- 5 **Large-scale deployment**: validate at 10K–100K nodes on real infrastructure
- 6 **Formal convergence guarantees**: for HEAL under partial participation and heterogeneous data
- 7 **Beyond supervised learning**: extend to unsupervised and reinforcement learning
- 8 **Incentive mechanisms**: reward participation, deter free-riding (potentially via blockchain/Lightning Network)

Thank you for your attention

Questions?

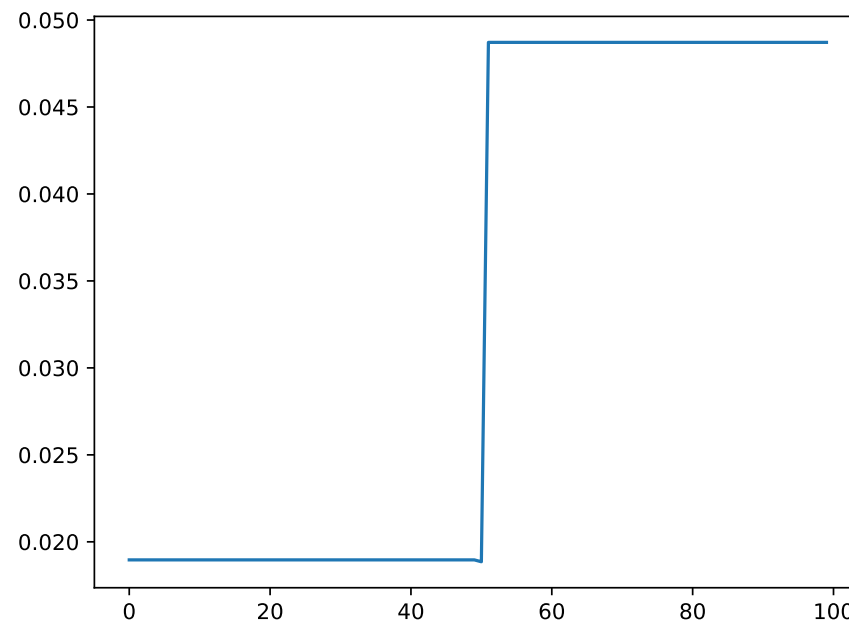
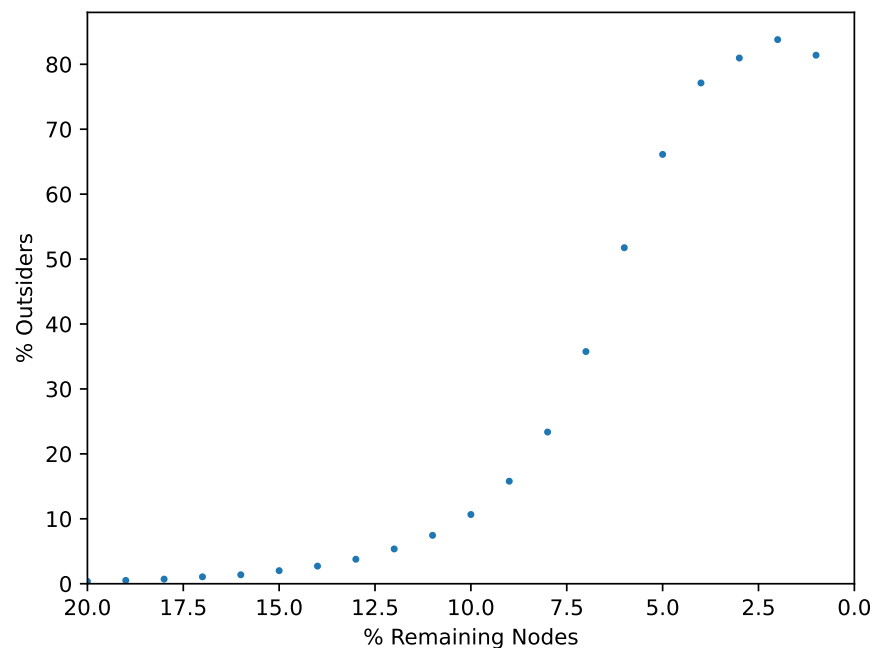
Appendix

International conferences:

- M. A. Legheraba et al., "A Brief Overview of Elevator," **ICDCN** 2024
- M. A. Legheraba et al., "Emergent Peer-to-Peer Multi-Hub Topology," **ICDCN** 2024
- M. A. Legheraba et al., "Lift: Byzantine-Resilient Hub Redistribution," **ICDCN** 2025 — **Outstanding Paper Award**
- M. A. Legheraba et al., "HEAL: Hub Enhanced Adaptive Learning," **IEEE IPDPS** 2025

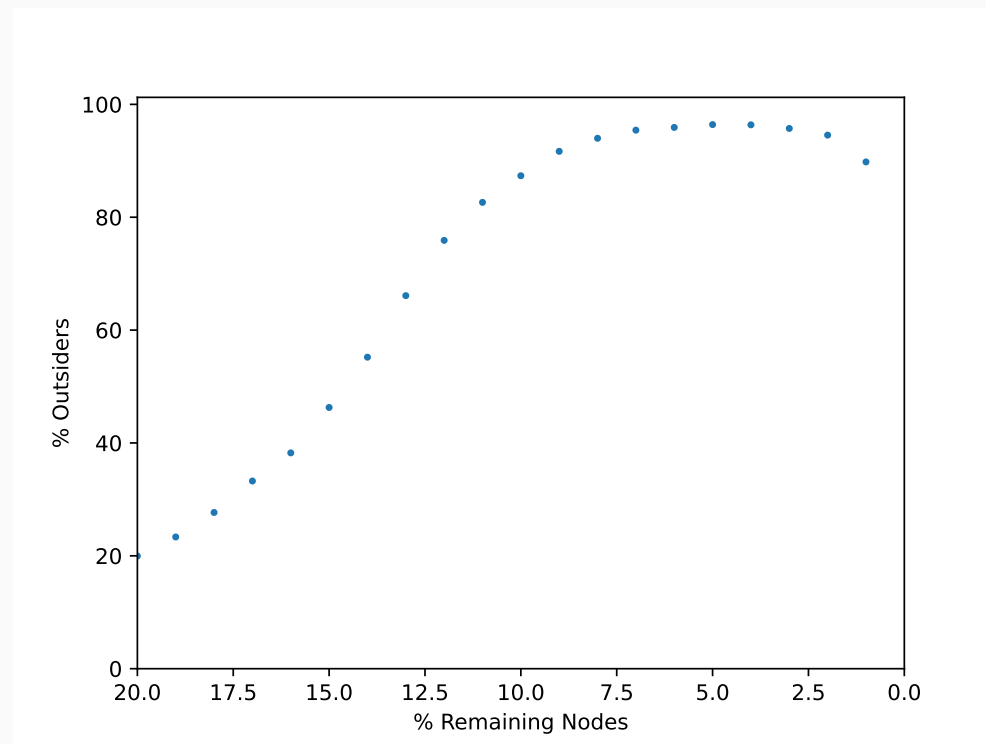
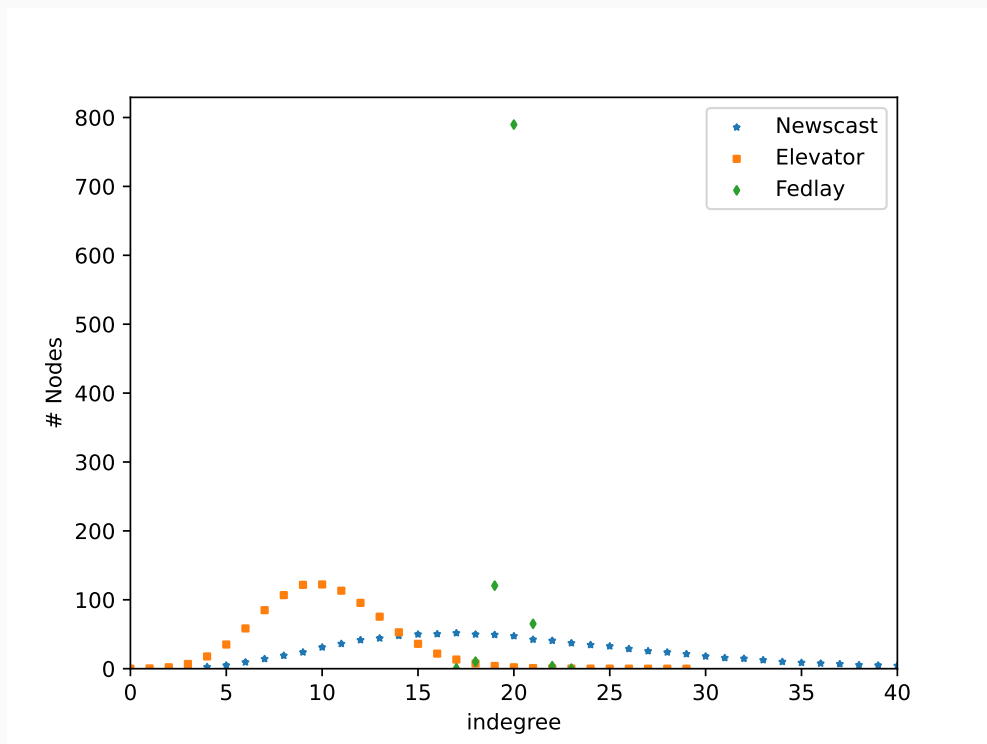
National conferences:

- M. A. Legheraba et al., "Nœuds hubs émergents," **Compas** 2025
- M. A. Legheraba et al., "Étoiles fédérées décentralisées," **Compas** 2025



Elevator under 20% crash scenario (1000 nodes, 100 experiments).

Largest connected component remains stable; clustering coefficient recovers within 2 cycles.



Left: indegree under hub crash — Elevator quickly re-elects new hubs.

Right: largest SCC under normal operation — Elevator maintains full connectivity.

Do-nothing attack

Byzantine returns empty cache → minimal impact on hub election

Non-coordinating attack

Each Byzantine inflates own ID → slight increase in Byzantine hub proportion

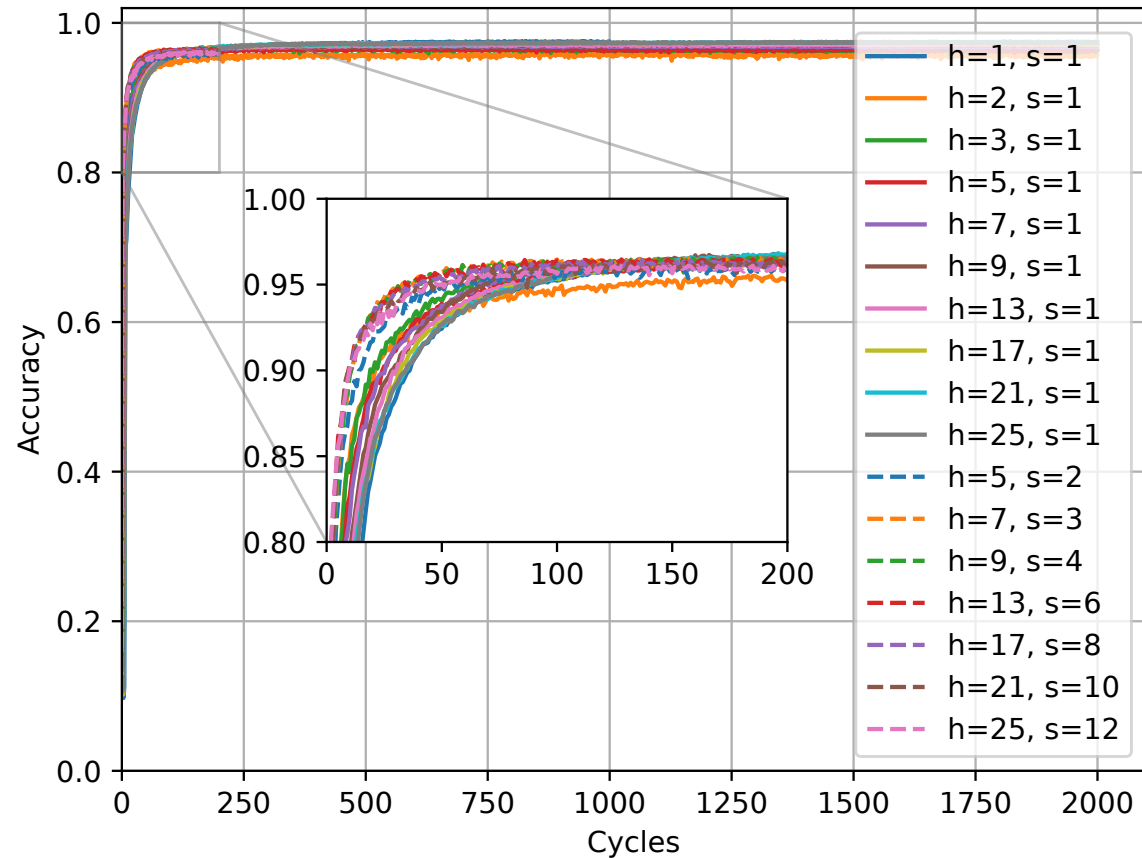
Coordinated attack

Byzantines share fake caches promoting each other → 2% sufficient to capture all hubs

Lift counter-measure

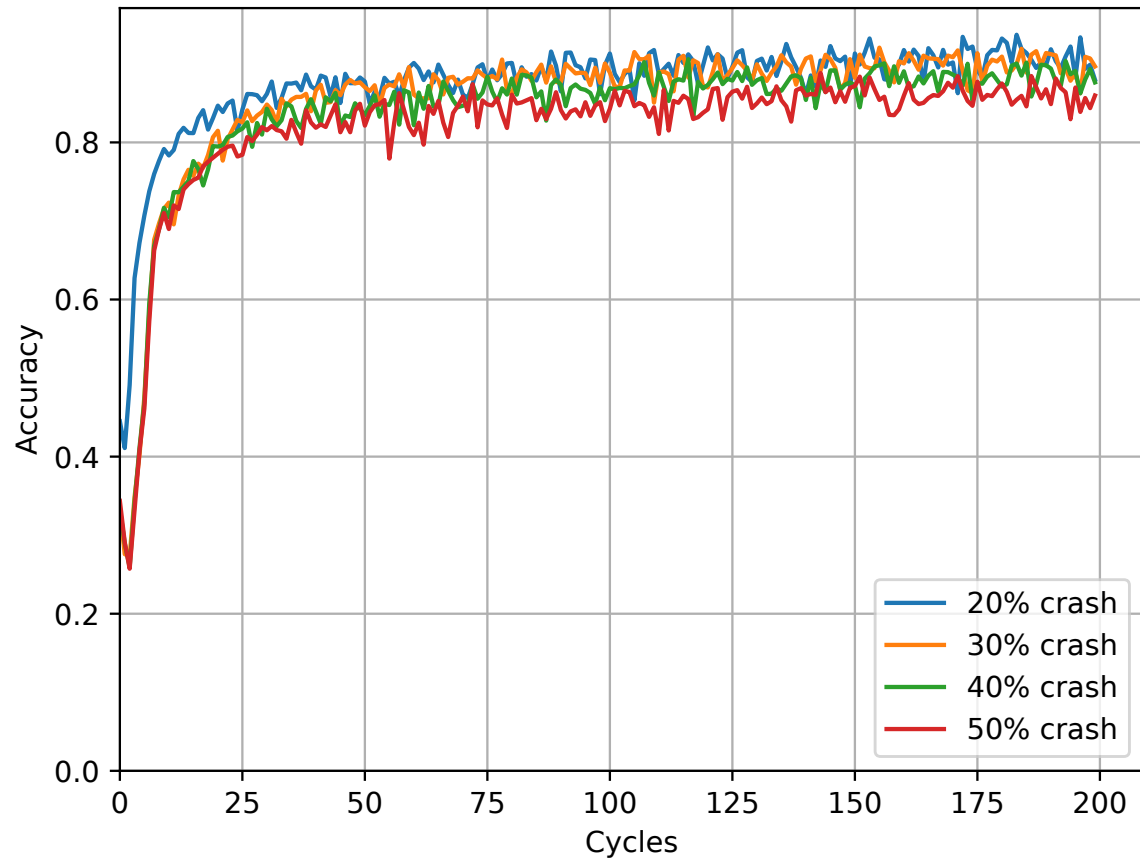
Deterministic redistribution via shared PRNG → tolerance extended to 10% collusion

HEAL: Impact of Number of Hubs



- Increasing h has minimal impact on final accuracy (hubs don't train, only aggregate)
- Sending to $s = \frac{h}{2}$ hubs slightly improves convergence speed
- $h = 5$, $s = 1$ offers a good balance between performance and resilience

HEAL: Crash of All Hubs



- 1 hub crash: no measurable impact (0.9638 accuracy at cycle 200)
- 5 hub crash (all hubs): **no measurable impact** (0.9629 accuracy)
- Elevator re-elects new hubs within 2 cycles → HEAL self-heals
- FL/Gaia: server crash → **training halts unconditionally**

Elevator & overlay evaluation: PeerSim (Java)

- Migrated from SVN to Git; introduced Gradle, Docker, CI/CD
- Parallelised simulation engine
- Implemented fault models, Byzantine attacks, overlay protocols

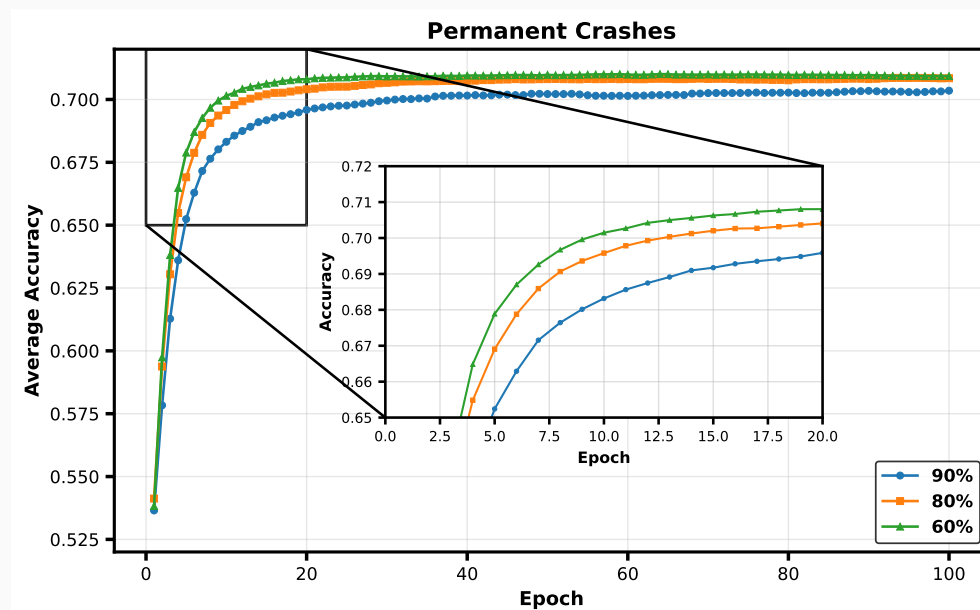
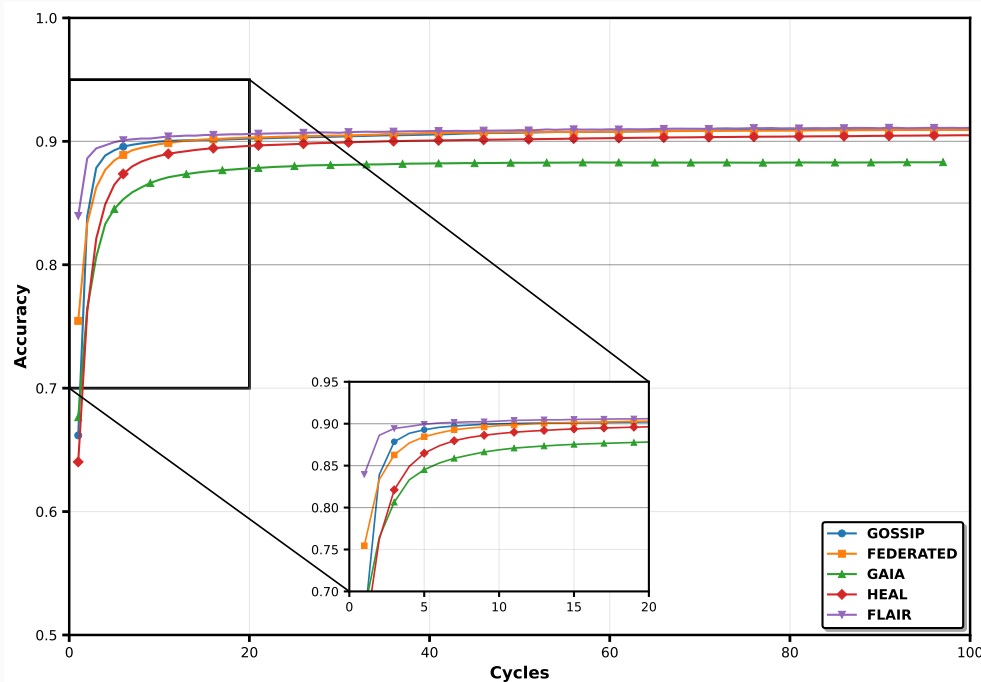
HEAL & learning evaluation: Gossipy + PeerSim (hybrid)

- PeerSim: dynamic topology (overlay layer)
- Gossipy: real PyTorch training (learning layer, not simulated)
- At each cycle: PeerSim graph → Gossipy determines aggregation partners

FLAIR evaluation: ns-3 (C++)

- Full network stack simulation (WiFi, propagation, interference)
- All baselines re-implemented in C++ for strict comparability

Ongoing: new unified Python simulator combining topology + learning + faults



Left: FLAIR vs baselines (100 nodes, fault-free). Right: under permanent fault scenario. Resource-aware CH election outperforms standard LEACH and Smart-LEACH.